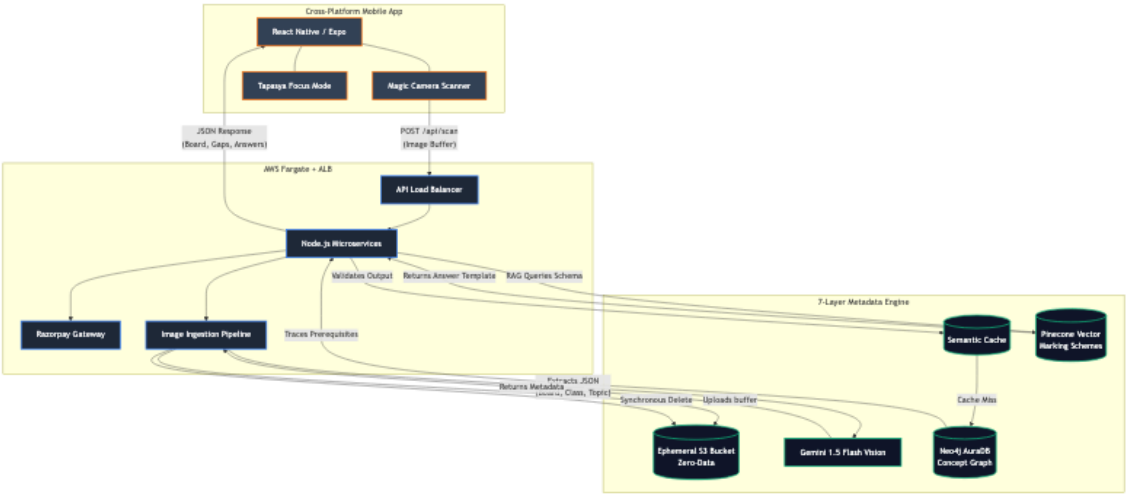
AI Pathshala

India's First Metadata-Driven Exam Success Engine

This document serves as the comprehensive architectural blueprint, local development runbook, and Technical Design Document (TDD) mapping for the AI Pathshala application.

Architecture Diagram



1. Core Architectural Constraints & Philosophy

AI Pathshala is built on three unbreakable engineering constraints to ensure total compliance with Indian educational standards and data privacy laws.

A. Zero-Chat Architecture

This is strictly an academic utility. We intentionally do not build conversational UIs to prevent student distraction. Responses are strictly structured visual metadata mapping (e.g., Mastery Maps, Smart Kunji model answers, and Tapasya focus modes).

B. Copyright-Safe Ephemeral Pipeline

To leverage the "non-expressive use" doctrine under Indian copyright law, no textbook scan image is ever permanently stored.

1. **Upload**: Images land in an AWS S3 `EphemeralScansBucket`.
2. **Inference**: A Vision Model (Gemini 1.5 Flash) holds the image buffer in memory strictly to extract structural JSON (Board, Class, Topic).
3. **Deletion**: The image is immediately purged from S3 via programmatic `DeleteObjectCommand` calls, backed by 1-day AWS lifecycle expiration failsafes. **Zero Data Stored.**

C. 7-Layer Metadata Constraint Engine

To enforce 0% AI hallucination, all LLM outputs must be constrained by our proprietary metadata stack:

1. **Curriculum**: Strict NCERT/State Board bounds.

2. **Concept Graph (Neo4j)**: Nodes mapping cross-grade prerequisite dependencies.
3. **Pedagogy**: Adjusting delivery based on Tapasya (Focus) vs Standard modes.
4. **Question Patterns**: Mapping against 2M, 3M, 5M marker schemas.
5. **Answer Templates (Pinecone)**: Vector DB RAG for exact official board marking schemes.
6. **Student State**: Delta progression tracking.
7. **Board Localization**: Language and terminology mapping (CBSE/ICSE).

---

2. Technology Stack & Infrastructure

Frontend: Cross-Platform Mobile

- **Framework**: React Native with Expo (Managed Workflow)
- **UI/UX**: "Serious Academic" Dark Mode interface (Slate 950 base, Orange 500 accents)
- **Navigation**: React Navigation (Native Stack)
- **Key Modules**: Magic Camera (AR Scanner), Smart Kunji (Solutions View), Mastery Map (Knowledge Graph), Tapasya Mode.

Backend: Node.js Microservices

- **Framework**: Express.js
- **Intelligence Integrations**:
  - `@google/genai` (Gemini Vision OCR)
  - `neo4j-driver` (AuraDB Graph queries)
  - `@pinecone-database/pinecone` (Vector RAG)
  - `ioredis` (Semantic caching)
  - `razorpay` (Payment server SDK)

Infrastructure: AWS CDK (Fargate)

- **Compute**: ECS Fargate Cluster providing autoscaling compute for API microservices, capable of handling 10,000 CCU.
- **Queueing**: SQS for asynchronous, detached grading operations.
- **Storage**: S3 with Lifecycle strict-delete constraints.
- **Routing**: Application Load Balancer securing the Fargate tasks.

---

3. Local Setup & Execution Guide

Follow these steps to spin up the entire application locally for development and testing.

Prerequisites

- Node.js (v18+)
- Local or Cloud Neo4j AuraDB instance active
- Pinecone Index created (`aipathshala-schemas` / 1536 dimensions)
- Gemini API Key

Step 1: Backend Setup

1. Open a terminal and navigate to the backend directory:

```
cd aipathshala/backend
```

2. Install dependencies:

```
npm install
```

3. Create a `.env` file in the `backend` root with the following keys:

```
PORT=3000
GEMINI_API_KEY=your_gemini_key
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=your_password
PINECONE_API_KEY=your_pinecone_key
REDIS_URL=redis://localhost:6379 # Optional if skipping cache
```

4. Start the Node backend in development watch mode:

```
npx ts-node-dev src/index.ts
```

(The backend is now listening on http://localhost:3000)

Step 2: Frontend Setup

1. Open a new terminal and navigate to the frontend directory:

```
cd aipathshala/frontend
```

2. Install dependencies:

```
npm install
```

3. If running on a physical Android/iOS Expo Go app, ensure your phone and computer are on the same Wi-Fi network. Update `src/services/api.ts` to replace `localhost` with your computer's local local IP address (e.g., `http://192.168.1.5:3000/api`).
4. Start the Metro Bundler:

```
npm run start
```

5. Press `w` to launch in a web browser, or scan the QR code with the **Expo Go** app on your physical device.

---

4. Technical Design Document (TDD) Specifications

A. Database Schemas & Data Constraints

1. Ephemeral S3 Bucket (`aipathshala-ephemeral-scans`)

- Object Key: `scan_<timestamp>_<uuid>.jpg`
- Content-Type: `image/jpeg` payload directly from Magic Camera.
- **Constraint**: Strict 1-Day AWS Lifecycle Policy + Synchronous `DeleteObject` via SDK.

2. Neo4j Concept Graph (Neo4j AuraDB)

- Label: `(:Concept {name: String, id: String, grade: String, board: String})`
- Edges: `[:REQUIRES]`, `[:LEADS_TO]` bridging grade-level prerequisites.

3. Pinecone Vector Schema (Index: `aipathshala-schemas`)

- Dimensions: 1536 (OpenAI Ada 002 compatible) or 768 (Gemini compatible).
- Metadata filtering schema: `{ topic: String, pattern: String (e.g. 5M), marks: Number }`

B. API Spec Mapping

1. `POST /api/scan`

- **Incoming**: `multipart/form-data` containing `image` buffer.
- **Controller**: Triggers Gemini 1.5 Flash Vision SDK to enforce the structural extraction, immediately dumps the memory buffer, then queries Neo4j + Pinecone based on the extracted overlay.
- **Response**:

```
{
  "success": true,
  "data": {
    "board": "CBSE",
    "class": "10",
    "topic": "Light Reflection",
    "masteryGaps": ["ConceptNode_Optics_Basic"],
    "answerTemplate": { "keyPoints": ["[+2] Refractive Index...", "[+1] Velocity..."] }
  }
}
```

2. `POST /api/pay/create-order`

- **Incoming**: `{ amount: Number, metadata: Object }`
- **Controller**: Triggers Razorpay internal SDK, creating an enforceable transaction sequence locking AI Pathshala features until webhook clearance.

C. Container Execution Detail

Fargate Task Deployment (`fargate-stack.ts`):

- 2x Fargate Microservice Tasks running `node:18-alpine` inside isolated VPC constraints.
- ALB (Application Load Balancer) mapping port 80/443 directly to Node Port 3000.
- SQS Queues acting as an asynchronous message bus if grading inference takes longer than standard HTTP latency.